

```

bool PrimeMC(unsigned int n,unsigned int k) { // 重复调用 k 次 Prime 算法的蒙特卡罗算法
    RandomNumber rnd;
    unsigned int a, result;
    bool composite = false;
    for (int i=1; i <= k; i++) {
        a = rnd.Random(n-3)+2;
        power(a, n-1, n, result, composite);
        if (composite || (result != 1))
            return false;
    }
    return true;
}

```

算法 PrimeMC 的错误概率不超过 $(1/4)^k$ 。这是一个保守的估计,实际使用的效果要好得多。

算法分析题 7

7-1 在实际应用中,常需模拟服从正态分布的随机变量,其密度函数为

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-a)^2}{2\sigma^2}}$$

式中, a 为均值, σ 为标准差。

如果 s 和 t 是 $(-1, 1)$ 中均匀分布的随机变量,且 $s^2+t^2<1$, 令

$$\begin{aligned}
 p &= s^2 + t^2 \\
 q &= \sqrt{(-2 \ln p) / p} \\
 u &= sq \\
 v &= tq
 \end{aligned}$$

则 u 和 v 是服从标准正态分布 ($\mu=0, \sigma=1$) 的两个互相独立的随机变量。

(1) 利用上述事实,设计一个模拟标准正态分布随机变量的算法。

(2) 将上述算法扩展到一般的正态分布。

7-2 设有一个文件含有 n 个记录。

(1) 试设计一个算法随机抽取该文件中 m 个记录。

(2) 如果事先不知道文件中记录的个数,应如何随机抽取其中的 m 个记录?

7-3 试设计一个算法,随机地产生范围在 $1 \sim n$ 的 m 个随机整数,且要求这 m 个随机整数互不相同。

7-4 设 X 是含有 n 个元素的集合,从 X 中均匀地选取元素。设第 k 次选取时首次出现重复。

(1) 试证明当 n 充分大时, k 的期望值为 $\beta\sqrt{n}$ 。其中, $\beta\sqrt{\pi/2}=1.253$ 。

(2) 由此设计一个计算给定集合 X 中元素个数的概率算法。

7-5 试设计一个随机化算法计算 $365!/340!365^{25}$, 并精确到 4 位有效数字。

7-6 一个问题是易验证的是指对该问题的给定实例的每个解,都可以有效地验证其正确性。例如,求一个整数的非平凡因子问题是易验证的,而求一个整数的最小非平凡因子就不是易验证的。在一般情况下,易验证问题未必是易解的。

(1) 给定一个解易验证问题 P 的蒙特卡罗方法, 设计一个相应的解问题 P 的拉斯维加斯算法。

(2) 给定一个解易验证问题 P 的拉斯维加斯算法, 设计一个相应的解问题 P 的蒙特卡罗算法。

7-7 用数组模拟有序链表的数据结构, 设计支持下列运算的舍伍德型算法, 并分析各种运算所需的计算时间:

(1) Predecessor 找出一给定元素 x 在有序集 S 中的前驱元素;

(2) Successor 找出一给定元素 x 在有序集 S 中的后继元素;

(3) Min 找出有序集 S 中的最小元素;

(4) Max 找出有序集 S 中的最大元素。

7-8 采用数组模拟有序链表的数据结构, 设计一个舍伍德型排序算法, 使算法最坏情况下的平均计算时间为 $O(n^3/2)$ 。

7-9 如果对于某个 n 值, n 后问题无解, 则算法将陷入死循环。

(1) 证明或否定下述论断: 对于 $n \geq 4$, n 后问题有解。

(2) 是否存在正数 δ , 使得对所有 $n \geq 4$ 算法成功的概率至少是 δ ?

7-10 假设已有一个算法 $\text{Prime}(n)$ 可用于测试整数 n 是否为一素数。还有一个算法 $\text{Split}(n)$ 可以实现对合数 n 的因子分割。试利用这两个算法, 设计一个对给定整数 n 进行因子分解的算法。

7-11 (1) 试证明下面的算法 Primality 能以 80% 以上的正确率判定给定的整数 n 是否为素数。另一方面, 举出整数 n 的一个例子, 表明算法对此整数 n 总是给出错误的解答, 进而说明该算法不是一个蒙特卡罗算法。

```
bool Primality(int n) {
    if (gcd(n, 30030) == 1)
        return true;
    else
        return false;
}
```

(2) 试找出上述算法 Primality 中可用于替换整数 30030 的另一个整数 (可使用大整数), 使得用此整数代替 30030 后, 算法的正确率提高到 85% 以上。

7-12 设 $\text{mc}(x)$ 是一致的 75% 正确的蒙特卡罗算法, 考虑下面的算法:

```
mc3(x) {
    int t, u, v;
    t = mc(x);
    u = mc(x);
    v = mc(x);
    if ((t == u) || (t == v))
        return t;
    return v;
}
```

(1) 试证明上述算法 $\text{mc3}(x)$ 是一致的 27/32 正确的算法, 因此是 84% 正确的。

(2) 试证明如果 $\text{mc}(x)$ 不是一致的, 则 $\text{mc3}(x)$ 的正确率有可能低于 71%。

7-13 设 $I=\{1, 2, \dots, n\}$, $S \subseteq I$ 是 I 的一个子集。 $mc(x)$ 是一个偏假 p 正确蒙特卡罗算法。该算法用于判定所给的整数 $1 \leq x \leq n$ 是否为集合 S 中的整数, 即 $x \in S$ 。设 $q=1-p$ 。由偏假算法的定义可知, 对任意 $x \in S$ 有 $\text{Prob}\{mc(x)=\text{true}\}=1$ 。当 $x \notin S$ 时, $\text{Prob}\{mc(x)=\text{true}\} \leq q$ 。考虑下面的产生 S 中随机元素的算法 GenRand 如下:

```
bool RepeatMC(int x, int k) {
    int i = 0;
    bool ans = true;
    while (ans && (i < k)) {
        i++;
        ans = mc(x);
    }
    return ans;
}
```

```
int GenRand(int n, int k) {
    RandomNumber rnd;
    int x = rnd.Random(n)+1;
    while (!RepeatMC(x, k))
        x = rnd.Random(n)+1;
    return x;
}
```

假设由语句 “ $x=\text{rnd.Random}(n)+1$;” 产生的整数 $x \in S$ 的概率为 r , 证明算法 GenRand 返回的整数不在 S 中的概率最多为

$$\frac{1}{1 + \frac{r}{1-r} q^{-k}}$$

7-14 设算法 A 和 B 是解同一判定问题的两个有效的蒙特卡罗算法。算法 A 是 p 正确偏真算法, 算法 B 是 q 正确偏假算法。试利用这两个算法设计一个解同一问题的拉斯维加斯算法, 并使所得到的算法对任何实例的成功率尽可能高。

7-15 考虑下面的无限循环算法:

```
void PrintPrimes(void) {
    cout << '2' << endl;
    cout << '3' << endl;
    int n = 5;
    while (true) {
        int m = floor(log(double(n)));
        if (PrimeMC(n, m))
            cout << n << endl;
        n = n+2;
    }
}
```

每个素数都会被上述算法输出。但是除了所有素数, 算法可能偶尔错误地输出某些合数。说明上述情况不太可能发生。或更精确, 证明上述算法错误地输出一个合数的概率小于 1%。

7-16 给定三个 $n \times n$ 矩阵 A 、 B 和 C , 下面的偏假 $1/2$ 正确的蒙特卡罗算法用于判定

$AB=C$ 。

```
bool Product(int **A, int **B, int **C, int n) { // 判定  $AB=C$  的蒙特卡罗算法
    RandomNumber rnd;
    int *x = new int [n+1];
    int *y = new int [n+1];
    int *z = new int [n+1];
    for (int i=1; i <= n; i++) {
        x[i] = rnd.Random(2);
        if (x[i] == 0)
            x[i] = -1;
    }
    Mult(B, x, y, n);
    Mult(A, y, z, n);
    Mult(C, x, y, n);
    for (int i=1; i <= n; i++)
        if (y[i] != z[i])
            return false;
    return true;
}
```

算法所需的计算时间为 $O(n^2)$ 。显然，当 $AB=C$ 时，算法 $\text{Product}(A, B, C, n)$ 返回 true。试证明当 $AB \neq C$ 时，算法返回值为 false 的概率至少为 1/2（提示：考虑矩阵 $AB-C$ 并证明当 $AB \neq C$ 时，将该矩阵各行相加或相减最终得到的行向量至少有一半是非零向量）。

算法实现题 7

7-1 模平方根问题。

问题描述：设 p 是奇素数， $1 \leq x \leq p-1$ ，如果存在一个整数 y ($1 \leq y \leq p-1$)，使得 $x \equiv y^2 \pmod{p}$ ，则称 y 是 x 的模 p 平方根。例如，63 是 55 的模 103 平方根。试设计一个求整数 x 的模 p 平方根的拉斯维加斯算法。算法的计算时间应为 $\log p$ 的多项式。

算法设计：设计一个拉斯维加斯算法，对于给定的奇素数 p 和整数 x ，计算 x 的模 p 平方根。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 2 个正整数 p 和 x 。

结果输出：将计算的 x 的模 p 平方根输出到文件 output.txt。当不存在 x 的模 p 平方根时，输出 0。

输入文件示例

input.txt

103 55

输出文件示例

output.txt

63

7-2 素数测试问题。

问题描述：试设计一个素数测试的偏真蒙特卡罗算法，对于测试的整数 n ，所述算法是一个关于 $\log n$ 的多项式时间算法。结合教材中素数测试的偏假蒙特卡罗算法，设计一个素数测试的拉斯维加斯算法（见算法分析题 7-14）。

算法设计：设计一个拉斯维加斯算法，对于给定的正整数，判定其是否为素数。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 p 。

结果输出：将计算结果输出到文件 output.txt。若正整数 p 是素数，则输出 “YES”，否则输出 “NO”。

输入文件示例

input.txt

103

输出文件示例

output.txt

YES

7-3 集合相等问题。

问题描述：给定两个集合 S 和 T ，试设计一个判定 S 和 T 是否相等的蒙特卡罗算法。

算法设计：设计一个拉斯维加斯算法，对于给定的集合 S 和 T ，判定其是否相等。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n ，表示集合的大小。接下来的 2 行，每行有 n 个正整数，分别表示集合 S 和 T 中的元素。

结果输出：将计算结果输出到文件 output.txt。若集合 S 和 T 相等则输出 “YES”，否则输出 “NO”。

输入文件示例

input.txt

3

2 3 7

7 2 3

输出文件示例

output.txt

YES

7-4 逆矩阵问题。

问题描述：给定两个 $n \times n$ 矩阵 A 和 B ，试设计一个判定 A 和 B 是否互逆的蒙特卡罗算法（算法的计算时间应为 $O(n^2)$ ）。

算法设计：设计一个蒙特卡罗算法，对于给定的矩阵 A 和 B ，判定其是否互逆。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n ，表示矩阵 A 和 B 为 $n \times n$ 矩阵。接下来的 $2n$ 行，每行有 n 个实数，分别表示矩阵 A 和 B 中的元素。

结果输出：将计算结果输出到文件 output.txt。若矩阵 A 和 B 互逆则输出 “YES”，否则输出 “NO”。

输入文件示例

input.txt

3

1 2 3

2 2 3

3 3 3

-1 1 0

1 -2 1

0 1 -0.666667

输出文件示例

output.txt

YES

7-5 多项式乘积问题。

问题描述：给定阶数分别为 n 、 m 和 $n+m$ 的多项式 $p(x)$ 、 $q(x)$ 和 $r(x)$ 。试设计一个判定 $p(x)q(x)=r(x)$ 的偏假 1/2 正确的蒙特卡罗算法，并要求算法的计算时间为 $O(n+m)$ 。

算法设计：设计一个蒙特卡罗算法，对于给定多项式 $p(x)$ 、 $q(x)$ 和 $r(x)$ ，判定 $p(x)q(x)=r(x)$ 是否成立。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 3 个正整数 n 、 m 、 l ，分别表示多项式 $p(x)$ 、 $q(x)$ 和 $r(x)$ 的阶数。接下来的 3 行，每行分别有 n 、 m 、 l 个实数，分别表示多项式 $p(x)$ 、 $q(x)$ 和 $r(x)$ 的系数。

结果输出：将计算结果输出到文件 output.txt。若 $p(x)q(x)=r(x)$ 成立，则输出“YES”，否则输出“NO”。

输入文件示例	输出文件示例
input.txt	output.txt
2 1 3	YES
1 2 3	
2 2	
2 6 10 6	

7-6 皇后控制问题。

问题描述：在 $n \times n$ 个方格组成的棋盘上的任一方格中放置一个皇后，该皇后可以控制其所在的行、列及对角线上的所有方格。对于给定的自然数 n ，在 $n \times n$ 个方格组成的棋盘上最少要放置多少个皇后才能控制棋盘上的所有方格，且放置的皇后互不攻击？

算法设计：设计一个拉斯维加斯算法，对于给定的自然数 n ($1 \leq n \leq 100$) 计算在 $n \times n$ 个方格组成的棋盘上最少要放置多少个皇后才能控制棋盘上的所有方格，且放置的皇后互不攻击。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n 。

结果输出：将计算的最少皇后数及最佳放置方案输出到文件 output.txt。文件的第 1 行是最少皇后数；接下来的 1 行是皇后的最佳放置方案。

输入文件示例	输出文件示例
input.txt	output.txt
8	5
	0 3 6 0 0 2 5 8

7-7 3-SAT 问题。

问题描述：SAT 的一个实例是 k 个布尔变量 $x_1, x_2, \dots, x_k$ 的 m 个布尔表达式 $A_1, A_2, \dots, A_m$ 。若存在各布尔变量 x_i ($1 \leq i \leq k$) 的 0, 1 赋值，使每个布尔表达式 A_i ($1 \leq i \leq m$) 都取值为 1，则称布尔表达式 $A_1 A_2 \dots A_m$ 是可满足的。

(1) 合取范式的可满足性问题 CNF-SAT

如果一个布尔表达式是一些因子和之积，则称为合取范式 (Conjunctive Normal Form, CNF)。这里的因子是变量 x 或 $\bar{x}$ 。例如， $(x_1 + x_2)(x_2 + x_3)(\bar{x}_1 + \bar{x}_2 + x_3)$ 就是一个合取范式，而 $x_1 x_2 + x_3$ 就不是合取范式。

(2) k -SAT

如果一个布尔合取范式的每个乘积项最多是 k 个因子的析取式，就称为 k 元合取范式，简记为 k -CNF。 k -SAT 问题是判定一个 k -CNF 是否可满足。特别地，当 $k=3$ 时，3-SAT 问题在 NP 完全问题树中具有重要地位。

(3) MAX-SAT

给定 k 个布尔变量 $x_1, x_2, \dots, x_k$ 的 m 个布尔表达式 $A_1, A_2, \dots, A_m$ ，求各布尔变量 x_i ($1 \leq i \leq k$) 的 0、1 赋值，使尽可能多的布尔表达式 A_i 取值为 1。

(4) Weighted-MAX-SAT

给定 k 个布尔变量 $x_1, x_2, \dots, x_k$ 的 m 个布尔表达式 $A_1, A_2, \dots, A_m$ ，每个布尔表达式 A_i 都有权值 w_i ，求各布尔变量 x_i ($1 \leq i \leq k$) 的 0、1 赋值，使取值 1 的布尔表达式权值之和达到最大。

算法设计：对于给定的带权 3-CNF，设计一个蒙特卡罗算法，使其权值之和尽可能大。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 2 个正整数 k 和 m ，分别表示变量数和布尔表达式数。接下来的 m 行中，每行有 5 个整数 $w, i, j, k, 0$ ，表示相应表达式的权值为 w ，表达式含的变量下标分别为 i, j, k ，行末以 0 结尾。下标为负数时，表示相应的变量为取反变量。

结果输出：将计算的最大权值输出到文件 output.txt。

输入文件示例	输出文件示例
input.txt	output.txt
5 3	26
9 3 1 4 0	
9 1 -5 3 0	
8 2 -5 1 0	

7-8 战车问题。

问题描述：在 $n \times n$ 格的棋盘上放置彼此不受攻击的车。按照国际象棋规则，车可以攻击与之处在同一行或同一列上的车。在棋盘上的若干个格中设置了堡垒，战车无法穿越堡垒攻击别的战车。对于给定的设置了堡垒的 $n \times n$ 格棋盘，设法放置尽可能多的彼此不受攻击的车。

算法设计：对于给定的设置了堡垒的 $n \times n$ 格棋盘，设计一个随机化算法，在棋盘上放置尽可能多的彼此不受攻击的车。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n 。接下来的 n 行中，每行有 1 个由字符 “.” 和 “X” 组成的长度为 n 的字符串。

结果输出：将计算的在棋盘上可以放置的彼此不受攻击的战车数输出到文件 output.txt。

输入文件示例	输出文件示例
input.txt	output.txt
4	6
....	
..X.	
.X..	
....	